PhaseA_Report

ECEN 723 Spring 2026: Project Phase A Report

Team ID: *17*
Group: i-group & v-group
Members: *Hyunsoo Lee, Joonhyun Choi, Vishnu Rajagopal, Stephen Muttathil*
Date: April 3, 2026

1. Code Structure

The implementation is organized into **three files**, cleanly separating shared state from each group's logic:

File	Role
`common_model.py`	Shared data structures, road graph, and transition predicates
`i_group_phaseA.py`	Infrastructure simulator: light arbitration and intersection access control
`v_group_phaseA.py`	Vehicle simulator: movement, routing, and tour completion

Each road between two nodes is modeled as **two independent directed segments** (e.g., `I00_to_I01` EAST and `I01_to_I00` WEST), with **slot 0** at the entry and **slot 29** at the intersection approach. The system advances in discrete **time steps**, where each step corresponds to a synchronized state transition across all vehicles and intersections.

2. Shared Data Model (`common_model.py`)

Key Data Classes

Class	Purpose
`VehicleState`	Encodes the full state of a vehicle: position (`segment` + `slot`), visit flags (B/C/D), and pending crossing intent
`IntersectionLightState`	Records which direction (if any) currently holds the green light at an intersection
`CrossingRequest`	A vehicle's **wait** on the intersection semaphore; declares intent to enter the critical section
`CrossingGrant`	The infrastructure's **signal**; releases the semaphore for exactly one vehicle per intersection per step
`Segment`	A directed road segment with `from_node` , `to_node` , `direction` , and `length_slots`

Transition Predicates

Three shared predicates enforce movement legality and are evaluated by both groups independently:

- `is_valid_crossing_transition` : outgoing segment must originate at the node where the incoming segment terminates; violations indicate an illegal state transition
- `is_u_turn_transition` : outgoing segment must not return to the node the vehicle came from; enforces the no-U-turn invariant
- `is_right_turn_transition` : outgoing direction must not be a right turn of the incoming direction (EAST→SOUTH, SOUTH→WEST, WEST→NORTH, NORTH→EAST); enforces the no-right-turn invariant

3. i-group: Infrastructure Simulator

3.1 Architecture

The i-group is built around two classes:

- `IntersectionController`: one instance per intersection; owns the light state and a per-direction **starvation counter** that tracks how long each direction has been denied access, ensuring the fairness property
- `InfrastructureSimulator`: the top-level coordinator; drives the per-step pipeline and accumulates a `SafetyReport` of all observed invariant violations

3.2 Algorithm

Each call to `step(vehicles)` executes the following pipeline:

Step 1: Identify waiting processes
`get_incoming_waiting_vehicles()` collects all vehicles blocked at **slot 29** with `request_crossing = True`; these are processes waiting to acquire their intersection's semaphore.

Step 2: Update the guard condition
For each intersection, `count_waiting_by_direction()` groups blocked vehicles by direction. `select_green_direction()` evaluates the guard condition using:

```
score(direction) = waiting_count(direction) + starvation_counter(direction)
```

The direction with the highest score is granted the green light. Non-winning directions with waiting vehicles increment their starvation counter; the winner's counter resets to 0. This scoring function guarantees the **starvation-freedom** property: no direction can be deferred indefinitely.

Step 3: Validate and build crossing requests
`build_crossing_requests()` validates each vehicle's request against all three transition predicates. Only requests that satisfy every predicate proceed; violations are logged immediately to the safety report.

Step 4: Acquire the semaphore
`select_one_grant()` filters valid requests to those satisfying the current guard condition (matching the green direction) and issues exactly **one grant** (a binary semaphore signal) per intersection. Ties are broken deterministically by `car_id` (lexicographic order), ensuring **deterministic arbitration**.

Step 5: Compute congestion map
Counts vehicles with `stopped = True` on segments feeding each intersection; a measure of system load used by the v-group's decision procedure.

Step 6: Verify safety invariants
`check_safety()` audits all active invariants and appends any violations to the `SafetyReport`.

3.3 Output (per step)

```
{
    "time_step": int,
    "lights": {"I00": "east" | "north" | "south" | "west" | None, ...},
    "crossing_grants": [{"intersection_id": str, "car_id": str, "granted": bool}, ...],
    "congestion_map": {"I00": int, ...},
    "safety_report": {
        "collisions": int,
        "red_light_violations": int,
        "simultaneous_green_violations": int,
        "invalid_grant_violations": int,
        "wrong_direction_violations": int,
        "u_turn_violations": int,
        "right_turn_violations": int,
```

```
    }
}
```

3.4 Safety Invariants Verified

Invariant	Mechanism
Mutual exclusion: no two vehicles share a (segment, slot)	`detect_collisions()` scans all position pairs each step
No red-light violation: only green-direction vehicles may cross	`check_safety()` verifies every grant aligns with the active guard condition
At most one green direction per intersection	Checked across all controllers; violation indicates a simultaneous-access fault
At most one crossing per intersection per step	`select_one_grant()` issues at most one semaphore signal per intersection
No U-turn state transition	`validate_request()` evaluates `is_u_turn_transition()`
No right-turn state transition	`validate_request()` evaluates `is_right_turn_transition()`
No illegal segment transition	`validate_request()` evaluates `is_valid_crossing_transition()`

4. v-group: Vehicle Simulator

4.1 Architecture

The v-group is built around two classes:

- `RoutePlanner`: a stateless decision procedure; selects the next segment given current position, target node, and the congestion map
- `VehicleSimulator`: maintains the global vehicle state; advances the system by one time step per call and independently tracks all invariant violations

4.2 Algorithm

Each call to `step(i_group_output)` executes in **two atomic phases**, mirroring a two-phase synchronization protocol:

Phase 1: `prepare_requests(congestion_map)`

For each vehicle:

1. **Blocking predicate** (`is_front_blocked()`): if the nearest vehicle ahead in the same segment is within **15 slots** (0.5 mile), the process blocks and clears its pending request.
2. **Internal state transition**: if not at slot 29, advance one slot.
3. **Terminal node**: record the visit and immediately execute the next segment transition chosen by the decision procedure.
4. **Intersection approach** (slot 29): invoke `build_crossing_request()`; the decision procedure selects a next segment and the vehicle issues a **wait** on the intersection semaphore by setting `request_crossing = True`.

A collision check (mutual exclusion audit) is run after all vehicles are processed.

Phase 2: `apply_i_group_output(lights, crossing_grants)`

For each vehicle holding a pending semaphore wait:

1. Check whether the i-group has issued a **signal** (grant) for this vehicle.
2. **If signaled**: evaluate all three transition predicates; if all pass, execute the crossing; the vehicle transitions to slot 0 of the new segment.
3. **If not signaled**: the vehicle remains blocked at slot 29, awaiting the next arbitration cycle.

After crossings are resolved, terminal visits are recorded, completed tours are detected, and any vehicle that has satisfied the liveness condition (visited all of B, C, D, and returned to A) is reset to begin a new tour.

Decision Procedure (Route Planning)

`choose_next_segment()` filters all candidate segments through the three transition predicates, then ranks the remainder by:

```
cost = manhattan_distance(next_node → target_node) × 2
     + congestion_penalty(next_node)
     + base_cost (10)
```

The **lowest-cost admissible candidate** is selected. This heuristic approximates a shortest-path search and is sufficient for Phase A; a complete decision procedure is planned for Phase B.

4.3 Output (per step)

```
{
    "time_step": int,
    "vehicles": {"car_id": {full state snapshot}, ...},
    "vehicles_for_i_group": {"car_id": {snapshot}, ...},
    "stats": {
        "completed_tours": int,
        "red_light_violations": int,
        "collisions": int,
        "illegal_direction_violations": int,
        "u_turn_violations": int,
        "right_turn_violations": int,
    }
}
```

4.4 Safety Invariants Verified

Invariant	Mechanism
Mutual exclusion: no two vehicles share a (segment, slot)	`check_collision()` audits all position pairs after each phase
No red-light violation	`apply_intersection_result()` checks the guard condition against the received grant
No U-turn state transition	`apply_intersection_result()` evaluates `is_u_turn_transition()`
No right-turn state transition	`apply_intersection_result()` evaluates `is_right_turn_transition()`
No illegal segment transition	`apply_intersection_result()` evaluates `is_valid_crossing_transition()`

5. i-group / v-group Integration Protocol

The two modules exchange the following signals each time step:

Direction	Signal	Semantics
v-group → i-group	`Dict[str, VehicleState]`	Full system state snapshot: vehicle positions, blocking status, semaphore wait requests
i-group → v-group	`lights`	Active guard condition: the direction whose semaphore wait may proceed
i-group → v-group	`crossing_grants`	Semaphore signal: names the one vehicle per intersection that may execute its crossing

Direction	Signal	Semantics
i-group → v-group	`congestion_map`	Load metric: stopped vehicle count per intersection, consumed by the decision procedure

The protocol guarantees that each step is processed against a **consistent global snapshot**: the v-group freezes vehicle state before sending it to the i-group, and no crossing is executed until the i-group's semaphore signals are received and applied.

6. Design Discussion

This section documents the rationale behind key implementation decisions through the lens of how the two groups negotiated their shared protocol and resolved design trade-offs.

Road representation

i-group: We need to evaluate the guard condition based on the direction a vehicle is approaching from. A single bidirectional segment object would require direction resolution at runtime on every state read.

v-group: Agreed. If every segment encodes exactly one direction, the decision procedure can filter candidates statically and the transition predicates remain simple boolean functions.

Decision: Each road is represented as two independent directed segments. Both groups share a single static segment map with no runtime ambiguity.

Intersection as a critical section: the semaphore model

v-group: We receive the `lights` output and know which direction satisfies the guard condition. Can we allow any vehicle in that direction to execute the crossing?

i-group: No; if two vehicles are both blocked at slot 29 on the same green direction, both satisfy the guard condition simultaneously. Without an explicit arbitration signal, both would attempt to enter the critical section in the same step, violating mutual exclusion and producing a collision.

v-group: So each intersection is effectively a critical section, and the `crossing_grant` is a **binary semaphore signal**; it names the one process that acquires the resource for this step.

i-group: Exactly. The `lights` output is the guard condition; the grant is the semaphore signal. A vehicle issues a **wait** by setting `request_crossing = True`; the i-group responds with a **signal** for exactly one vehicle per intersection. Both groups then independently verify that no invariant was violated; the i-group checks that only the granted vehicle executed the crossing; the v-group checks that it only crossed when it held a valid signal.

Decision: Each intersection is modeled as a binary semaphore. The i-group controls the semaphore and issues at most one signal per step. Both groups audit the same events independently; their violation counts must match.

Starvation-freedom in light selection

v-group: A purely queue-length-based arbitration policy can permanently favor high-traffic directions. Vehicles on low-traffic approaches would never acquire the semaphore; the system would exhibit **starvation**.

i-group: We address this with a starvation counter per direction. Each step a direction is denied the semaphore, its counter increments. The selection function scores each direction as `waiting_count + starvation_counter`, so a direction that has waited long enough will eventually outrank a busier one. This guarantees the **starvation-freedom** property: every waiting vehicle is eventually granted access.

Decision: The scoring function combines queue length with accumulated wait time. The counter resets on grant and increments otherwise, enforcing fairness without requiring a full scheduling proof at this phase.

Two-phase synchronization protocol

i-group: We require a **consistent global snapshot** of vehicle positions before computing lights and grants. If vehicles were still transitioning when we read state, our guard condition evaluation would be based on a partially-updated system state.

v-group: We enforce this with a two-phase step. Phase 1 advances all vehicles within their segments and freezes the state into a snapshot; this is what we send to you. Phase 2 applies your semaphore signals to resolve intersection crossings. The two phases are never interleaved; you always observe a stable state.

Decision: `prepare_requests()` produces a consistent snapshot before any semaphore signal is consumed. `apply_i_group_output()` applies signals only after the snapshot has been transmitted. This mirrors a two-phase commit protocol for shared state.

15-slot observability limit

i-group: The specification constrains a vehicle's observable state: it can detect another vehicle ahead only within 0.5 mile (15 slots) and only if no third vehicle is between them.

v-group: The original blocking predicate was over-conservative; it treated any vehicle ahead in the same segment as blocking, regardless of distance. A vehicle 25 slots ahead is outside the observable range and should not affect the blocking decision.

Decision: `is_front_blocked()` identifies the nearest vehicle ahead in the same segment and evaluates the predicate only if the gap is at most 15 slots. This correctly models the bounded observability constraint from the specification.

Deterministic arbitration for semaphore signals

v-group: When multiple vehicles satisfy the guard condition at the same intersection, the arbitration must be deterministic to allow reproducible verification runs.

i-group: We order candidates lexicographically by `car_id` and select the first. This requires no additional state, produces identical outputs for identical inputs, and makes counterexample traces fully reproducible; a property that will be important in Phase C verification.

Decision: Semaphore signals are awarded to the lexicographically first eligible `car_id`.

Right-turn prohibition: proactive vs. defensive enforcement

v-group: The no-right-turn constraint is enforced proactively in the decision procedure; a right-turn segment is never admitted as a candidate. The invariant is upheld before a semaphore wait is even issued.

i-group: We apply a second, defensive check during request validation and in the safety audit. If a right-turn request arrives, due to a bug or an unanticipated code path, it is rejected and logged as a violation. This layered enforcement follows a **defense-in-depth** approach: the v-group prevents violations by construction; the i-group detects them if they occur.

Decision: Right-turn transitions are excluded from the decision procedure and independently audited by both groups' safety checkers.

7. Test Results

Three test scenarios are presented: a standalone i-group test, a standalone v-group test, and an end-to-end integrated run of both groups together. All outputs shown are actual program output.

7.1 i-group Standalone Test

Setup: Three vehicles across two intersections to exercise semaphore arbitration and the mid-segment exclusion rule.

- `car_1`: segment `A_to_I00`, slot 29, eastbound, requesting entry to `I00_to_I01`
- `car_2`: segment `I10_to_I11`, slot 29, eastbound, requesting entry to `I11_to_I12`
- `car_3`: segment `I01_to_I11`, slot 12, mid-segment, no semaphore wait

Actual output (Step 1):

```
{'time_step': 1,
 'lights': {'I00': 'east', 'I01': None, 'I02': None, 'I10': None,
            'I11': 'east', 'I12': None, 'I20': None, 'I21': None, 'I22': None},
 'crossing_grants': [
     {'intersection_id': 'I00', 'car_id': 'car_1', 'granted': True},
     {'intersection_id': 'I11', 'car_id': 'car_2', 'granted': True}
 ],
 'congestion_map': {'I00': 1, 'I01': 0, 'I02': 0, 'I10': 0,
                    'I11': 1, 'I12': 0, 'I20': 0, 'I21': 0, 'I22': 0},
 'safety_report': {
     'collisions': 0, 'red_light_violations': 0,
     'simultaneous_green_violations': 0, 'invalid_grant_violations': 0,
     'wrong_direction_violations': 0, 'u_turn_violations': 0,
     'right_turn_violations': 0
 }}
```

Observations:

- Guard condition is set to `east` at exactly those intersections where an eastbound vehicle is blocked at slot 29; all other lights remain `None`
- Exactly one semaphore signal is issued per active intersection; mutual exclusion holds
- `car_3` at slot 12 is correctly excluded from arbitration; the semaphore wait precondition (slot 29, `request_crossing = True`) is not satisfied
- All invariant violation counters are zero

7.2 i-group: Starvation-Freedom Test

Setup: Two vehicles held permanently at the same intersection from competing directions to verify that the starvation counter prevents one direction from monopolizing the semaphore.

- `car_east`: segment `I10_to_I11`, slot 29, eastbound, requesting `I11_to_I12`
- `car_north`: segment `I21_to_I11`, slot 29, northbound, requesting `I11_to_I01`

Actual output (Steps 1-4):

```
Step 1: light=north  grants=[{'intersection_id': 'I11', 'car_id': 'car_north', 'granted': True}]
        starvation_east=1  starvation_north=0
Step 2: light=east   grants=[{'intersection_id': 'I11', 'car_id': 'car_east',  'granted': True}]
        starvation_east=0  starvation_north=1
Step 3: light=north  grants=[{'intersection_id': 'I11', 'car_id': 'car_north', 'granted': True}]
```

```
        starvation_east=1  starvation_north=0
Step 4: light=east   grants=[{'intersection_id': 'I11', 'car_id': 'car_east',  'granted': True}]
        starvation_east=0  starvation_north=1
```

Observations:

- The light alternates between the two directions because equal queue lengths make the starvation counter the tiebreaker
- Each denied direction increments its counter by 1; the winning direction resets to 0
- Neither direction is ever denied for two consecutive steps; starvation-freedom holds

7.3 i-group: Illegal Transition Rejection Test

Setup: Three vehicles at separate intersections: one valid straight crossing, one U-turn attempt, and one right-turn attempt.

- `car_ok`: `A_to_I00`, slot 29, eastbound, requesting `I00_to_I01` (valid straight crossing)
- `car_uturn`: `I00_to_I01`, slot 29, eastbound arriving at I01, requesting `I01_to_I00` (U-turn back west)
- `car_right`: `I21_to_I11`, slot 29, northbound arriving at I11, requesting `I11_to_I12` (right turn: north to east)

Actual output (Step 1):

```
lights:  {'I00': 'east', 'I01': 'east', 'I11': 'north'}
grants:  [{'intersection_id': 'I00', 'car_id': 'car_ok', 'granted': True}]
safety:  {
    'collisions': 0, 'red_light_violations': 0,
    'simultaneous_green_violations': 0, 'invalid_grant_violations': 0,
    'wrong_direction_violations': 0,
    'u_turn_violations': 1, 'right_turn_violations': 1
}
```

Observations:

- `car_ok` receives the only grant; its crossing passes all three transition predicates
- `car_uturn` is rejected by `validate_request()` and logged as a U-turn violation; no grant is issued at I01
- `car_right` is rejected and logged as a right-turn violation; no grant is issued at I11
- Rejection occurs before `select_one_grant()`, so neither illegal vehicle ever competes for the semaphore

7.4 v-group Standalone Test

Setup: Two vehicles starting at `A_to_I00`, slot 0, to exercise the blocking predicate and the two-phase protocol in isolation (no i-group input).

Actual output:

```
Step 1 (no i-group signals):
  car_1: seg=A_to_I00  slot=1  stopped=False  req=False
  car_2: seg=A_to_I00  slot=0  stopped=True   req=False
  stats: {completed_tours:0, red_light_violations:0, collisions:0,
          illegal_direction_violations:0, u_turn_violations:0, right_turn_violations:0}

Step 2 (i-group signal: I00=east, car_1 granted):
  car_1: seg=A_to_I00  slot=2  stopped=False  req=False
  car_2: seg=A_to_I00  slot=0  stopped=True   req=False
  stats: all zeros
```

Observations:

- `car_1` advances: slot 0 to 1 to 2 across two steps
- `car_2` blocks at slot 0 because `car_1` is within 15 slots ahead in the same segment; the blocking predicate fires
- The grant for `car_1` at I00 in Step 2 does not produce a crossing because `car_1` has not yet reached slot 29; the signal is correctly ignored

7.5 End-to-End Integration Test

Setup: Both simulators running together in a closed loop for 50 steps. The v-group snapshot is forwarded to the i-group each step and the i-group lights and grants are fed back to the v-group. Two vehicles start at `A_to_I00`, slot 0.

Selected step trace (actual output):

```
Step 1:
  car_1: seg=A_to_I00   slot=1   stopped=False  req=False
  car_2: seg=A_to_I00   slot=0   stopped=True   req=False
  lights: {}    grants: []

Step 29:
  car_1: seg=A_to_I00   slot=29  stopped=False  req=False
  car_2: seg=A_to_I00   slot=14  stopped=False  req=False
  lights: {}    grants: []
  (car_2 is now free; car_1 is exactly 15 slots ahead, at the visibility boundary)

Step 30:  [car_1 issues semaphore wait; i-group responds with grant]
  car_1: seg=A_to_I00   slot=29  stopped=True   req=True   next=I00_to_I01
  car_2: seg=A_to_I00   slot=14  stopped=True   req=False
  lights: {'I00': 'east'}
  grants: [{'intersection_id': 'I00', 'car_id': 'car_1', 'granted': True}]

Step 31:  [car_1 executes crossing; state transition to new segment]
  car_1: seg=I00_to_I01  slot=0   stopped=False  req=False
  car_2: seg=A_to_I00    slot=14  stopped=True   req=False
  lights: {'I00': 'east'}
  grants: [{'intersection_id': 'I00', 'car_id': 'car_1', 'granted': True}]

Step 48:  [car_2 has also crossed; both vehicles on I00_to_I01]
  car_1: seg=I00_to_I01  slot=17  stopped=False  req=False
  car_2: seg=I00_to_I01  slot=0   stopped=False  req=False
  lights: {'I00': 'east'}
  grants: [{'intersection_id': 'I00', 'car_id': 'car_2', 'granted': True}]

All violation counters: zero across both groups for all 50 steps
```

Observations:

- `car_1` reaches slot 29 after 29 steps (one slot per step, unblocked once `car_2` falls 15 slots behind)
- At step 30, the crossing request fires and the i-group immediately sets the guard condition and issues a grant in the same step
- At step 31, `car_1` transitions atomically to slot 0 of `I00_to_I01`; one step per crossing, exactly as required
- `car_2` follows the same path 17 steps later; the i-group issues a separate independent grant
- Both groups independently count zero violations across all 50 steps; their safety audit results agree

8. Source Code

Source code is included in the submitted repository:

- `common_model.py`: shared data model, road graph, and transition predicates

- `i_group_phaseA.py` : infrastructure simulator with semaphore-based intersection control
- `v_group_phaseA.py` : vehicle simulator with two-phase synchronization protocol